

8. Serialización y Networking

Last edited time	October 14, 2026 9:57 PM
Practica	☐
Teoria	☑

⚠️

- **Lubuntu**
  - ¡Evitar ocupar estos puertos!
    1234, 5000, 7860, 11434, 8888
  - Ocupar puertos como 12345, 8080, 9000, etc.

Serialización JSON

- La **serialización** transforma el estado de un objeto a una representación que puede guardarse o transmitirse.
  - Dicha representación puede ser texto (como JSON) o una secuencia binaria.
  - Para enviar texto por una red es necesario **codificarlo** a bytes (ej. **UTF-8**).
- La **deserialización** corresponde al proceso inverso, es decir, convertir el objeto recibido en JSON (por ejemplo) a un objeto funcional.

JSON

- **JSON: J**ava**S**cript **O**bject **N**otation
  - Formato de texto estándar para intercambiar datos entre programas.
- El módulo `json` permite serializar y deserializar objetos.

▼ `json` transforma los siguientes tipos

Python	JSON
`dict`	objeto
`list, tuple`	arreglo
`str`	string
`int, float`	número
`True, False`	`true, false`
`None`	`null`

👁️

1. **Por defecto**, no se pueden serializar funciones, conjuntos ni instancias de clases personalizadas
2. En el caso de las listas y las tuplas, ambas se serializan en formato de un **arreglo**. Sin embargo, cuando se deserializa un arreglo, este retorna una **lista** (por defecto).

Funciones de `json`

- `dumps` serializa un objeto y retorna un `str`

  ▼ `indent=None`

  - Acepta un entero positivo o una string, para que los elementos y objetos de un arreglo sean impresos con esa indentación.

▼ `ensure_ascii=True`

- El resultado va a tener todos los caracteres que no son ASCII en otro formato (escapados).
- Si está en `False` , la mayoría de los caracteres se mostrara como son.

- `loads` deserializa un `str` con JSON
- `dump(objeto, archivo)` serializa `objeto` en `archivo` , es decir, escribe su representación JSON.
- `load(archivo)` deserializa un `archivo` JSON, o sea, lo lee y retorna una representación en Python.

- En caso que no se cumple la sintaxis JSON, `loads` y `load` levantan un `json.JSONDecodeError`
- Como JSON ocupa unicode, en el caso de ocupar `dump` y `load` , conviene agregar `encoding="utf-8"` al `open()` del archivo.

Personalizar la (de)serialización

- Es posible serializar una instancia de una clase usando el diccionario que retorna el método `__dict__`
  - El diccionario contiene los pares atributos-valores de la instancia

▼ Alternativamente, se puede personalizar la serialización con una subclase de `json.JSONEncoder` y la deserialización con una función mediante `object_hook`

- La subclase se le pasa como argumento a `dump` / `dumps` con `cls=SubclaseEncoder`
- La función se le pasa como argumento a `load` / `loads` con `object_hook=funcion_deserializadora`

- La subclase de `json.JSONEncoder` debe sobrescribir el método `default(obj)`
  - Si `obj` es de la instancia que buscamos, retornamos un diccionario con sus atributos-valores. En caso contrario, se retorna `super.default(obj)` para que levante un `TypeError` .
  - En el diccionario puede incluirse también un ítem como **"tipo": "nombre de la clase",** para poder reconocer más adelante que un diccionario deserializado corresponde a una clase.
  - Se puede ocupar la misma subclase para serializar más de una clase personalizada.
- `object_hook` recibe el resultado de cada objeto JSON ({...}) convertido inicialmente a `dict` → no se ejecuta para arreglos, strings, números, booleanos ni `null`
  - La función para deserializar que se le pasa debe reconocer solo los diccionarios que le corresponden y retornar sin cambios los demás.

▼ Ejemplo de función

```
def funcion_mascota_hook(diccionario: dict):
    if diccionario.get("tipo") != "Mascota":
        return diccionario
    return Mascota(
        nombre=diccionario["nombre"],
        edad=diccionario["edad"],
        especie=diccionario["especie"],
        id_=diccionario["Mascota_id"],
    )
```

▼ `object_hook` se aplica a **cada objeto JSON,** comenzando por los objetos más internos. Por esto, el resultado de una llamada puede formar parte del diccionario que reciba una llamada posterior.

```
def funcion_lista_hook(diccionario):
    """Esta función transforma un diccionario en un número"""
    print(f"Me llegó el diccionario: {diccionario}")
    valor = 33 if len(diccionario) > 1 else 42
    print(f"Lo transformaré en {valor}\n")
```

```
    return valor


json_string = '{"a": {"b": 1, "c": [2, 3, {}], "d": null, "e": {"f": true}}}'
datos = json.loads(json_string, object_hook=funcion_lista_hook)

print(datos)
# ...
# ...
# 42
```

Conceptos básicos de Networking

- Las redes de datos permiten que dos procesos se comuniquen entre sí, tanto si están en máquinas diferentes o en dispositivos distintos.
- Componentes esenciales para la comunicación entre computadores
  - Medio para transmitir señales
    - Ethernet, fibra óptica, Wi-Fi o redes móviles.
  - Identificación de la **interfaz** de destino

    ▼ Dirección IP
    - Un dispositivo puede tener varias interfaces y, por ende, múltiples direcciones IP.
    - IPv4 utiliza 32 bits (2^{32} combinaciones)
    - IPv6 utiliza 128 bits (2^{128} combinaciones)
  - Identificación del **proceso** de destino

    ▼ Puerto
    - Identifican puntos de comunicación de la capa de transporte.
    - Tienen 16 bits y valores entre 0 y 65535.

    ▼ Asignación depende del protocolo y dirección local

Puerto	Protocolo o servicio
21	FTP, conexión de control
22	SSH
25	SMTP
53	DNS
80	HTTP
110	POP3
143	IMAP
443	HTTPS

En FTP, el puerto 21 se usa para control; el 20 puede utilizarse para datos en modo activo.

> 👁️ TCP y UDP pueden usar el mismo número de puerto simultáneamente.

  - **Protocolo de transporte**

    ▼ TCP
    - **Transmission Control Protocol**
    - Ofrece a la aplicación un flujo **ordenado** y **confiable** de bytes.
    - No conserva los límites de los mensajes de la aplicación.
    - TCP establece una conexión mediante un *handshake*, utiliza números de secuencia, ACK acumulativos, retransmisiones y mecanismos de control de flujo y congestión. Durante el

handshake pueden negociarse opciones como MSS, escalamiento de ventana y SACK; no se negocia directamente la memoria de la aplicación.

- **HTTP (/1.1 y /2)** y **SMTP** suelen usar TCP.

▼ UDP

- **User Datagram Protocol**
- Envía **datagramas independientes** sin establecer una conexión previa.
- Conserva sus límites, pero **no garantiza entrega, orden ni retransmisión.**
- Se usa en DNS, telefonía y video en tiempo real, juegos y como base de QUIC.
- **HTTP/3** utiliza QUIC sobre UDP.

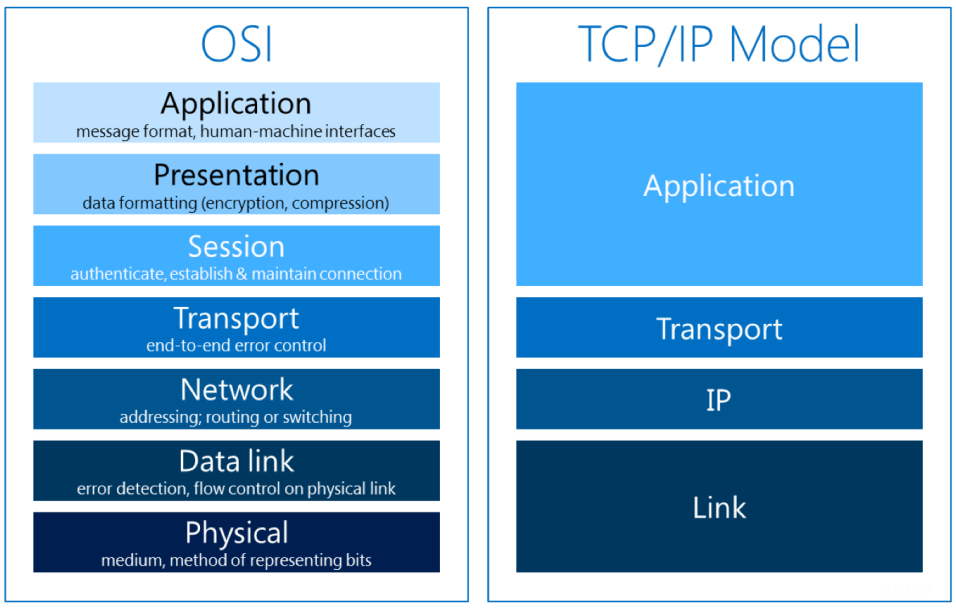
◦ **Protocolo de aplicación**, que define significado y formato de los mensajes.

▪ HTTP, DNS o SMTP

👁️ **DNS (*Domain Name System*)** convierte nombres de dominio (ej. google.com) a direcciones IP mediante un sistema distribuido y jerárquico.

📌 La información se encapsula por capas: TCP usa **segmentos**, UDP **datagramas** y la capa IP los transporta en **paquetes IP**. Los puertos pertenecen a TCP o UDP; las direcciones pertenecen a IP.

Encapsulamiento



- Cada capa ofrece servicios a la capa superior y agrega información de control.
- Nuestros programas operan en la capa de **aplicación** y utilizan servicios del SO, como sockets TCP o UDP. Los datos se encapsulan al bajar por la pila y se desencapsulan en el destino.

Arquitectura cliente-servidor

- Un **proceso** servidor ofrece una funcionalidad y uno o más procesos **cliente** la consumen.
- Cliente y servidor pueden estar en computadores distintos o en la misma máquina (`localhost`)
- TCP → cliente **inicia una conexión** hacia la dirección y puerto donde escucha el servidor.
- UCP → cliente **envía datagramas** al puerto del servidor, sin una conexión TCP previa.

Conceptos básicos de Webservices

- Un **servicio web** permite que programas intercambien información mediante tecnologías de la web, usualmente HTTP.
  - Un sitio o aplicación web suele entregar HTML, CSS y JavaScript al usuario del navegador.
  - Una **API web expone operaciones y datos para otras aplicaciones**, usualmente JSON.

- Un mismo servidor puede ofrecer páginas web y endpoints de una API.

Application Programming Interface: API

- Define cómo otros programas interactúan con un componente o servicio.
- Puede contener funciones, métodos o endpoints web.
- Una API web documenta su URL base, endpoints, métodos HTTP, parámetros, cuerpos, autenticación, respuestas y códigos de estado.
- Un navegador en un cliente HTTP.

📌 Un **endpoint** (o punto de extremo) de una API es una dirección URL específica a la que un cliente (como una aplicación móvil, sitio web o software) envía solicitudes para consultar, crear, modificar o eliminar datos en un servidor.
- Es la puerta de entrada a un recurso concreto dentro de un sistema web.

▼ Ejemplo

Si un servidor administra cuentas de usuario, sus endpoints se verían de la siguiente forma:

Método HTTP	Endpoint (Ruta)	Acción solicitada
GET	`/api/usuarios`	Obtener la lista completa de usuarios
GET	`/api/usuarios/45`	Obtener la información del usuario con ID 45
POST	`/api/usuarios`	Crear un nuevo usuario
PUT	`/api/usuarios/45`	Actualizar los datos del usuario con ID 45
DELETE	`/api/usuarios/45`	Eliminar al usuario con ID 45

Uniform Resource Locator (URL)

- Indica cómo localizar un recurso.
- Se puede llegar a un mismo recurso a través de URLs distintas.

```
https://api.ejemplo.com:8080/users/42/posts?page=2&limit=10#comentarios
└───┘ └──────────────┘└──┘└──────────┘└──────────┘└──────┘
esquema      host       puerto    ruta        query string    fragmento
```

Componente	Ejemplo	Descripción
Esquema	`https`	Mecanismo de acceso.
Host	`api.ejemplo.com`	Dominio o dirección IP.
Puerto	`8080`	Puerto opcional; HTTP usa 80 y HTTPS 443 por defecto.
Ruta	`/users/42/posts`	Recurso dentro del servicio.
Query string	`page=2&limit=10`	Parámetros después de `?`.
Fragmento	`#comentarios`	Sección del documento; no se envía al servidor.

HTTP (Hypertext Transfer Protocol)

- Familia de protocolos de solicitud y respuesta.
- Es **stateless:** cada solicitud tiene significado por sí misma.
- La aplicación puede guardar usuarios, compras y sesiones.

▼ **Métodos HTTP**

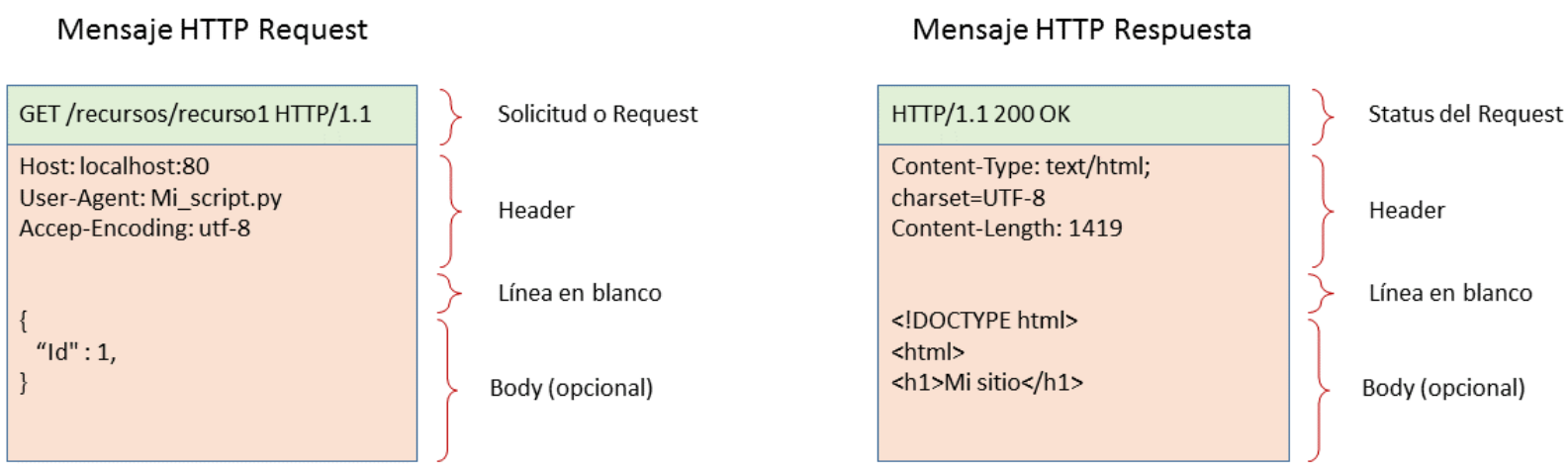
- `GET` solicita una representación y debe ser seguro
- `HEAD` es como `GET` , pero sin contenido de respuesta.
- `POST` pide procesar el contenido; frecuentemente crea recursos, pero no se limita a eso.
- `PUT` crea o reemplaza el estado del recurso identificado.
- `PATCH` aplica cambios parciales
- `DELETE` solicita eliminar la asociación con el recurso
- `OPTIONS` consulta opciones
- `CONNECT` crea un túnel
- `TRACE` realiza diagnóstico

▼ Códigos de estado

Rango	Categoría
1xx	Informacional
2xx	Éxito
3xx	Redirección
4xx	Error del cliente
5xx	Error del servidor

▼ Mensajes HTTP

Solicitudes y respuestas contienen una línea inicial, campos de encabezado y contenido opcional.



`Content-Type` describe el formato

TLS: Transport Layer Security

- TLS crea un canal seguro
  - `https://` → HTTP se comunica mediante TLS
    - HTTP/1.1 y HTTP/2 normalmente usa TLS con TCP.
    - HTTP/3 usa TLS 1.3 dentro de QUIC sobre UDP
- TLS proporciona confidencialidad, integridad y autenticación del **servidor**.
  - La autenticación del cliente es opcional.

▼ Handshake TLS 1.3

1. Cliente ofrece versiones, algoritmos, grupos y una participación de clave.
2. Servidor selecciona parámetros y responde con su participación.
3. Ambos derivan secretos, normalmente mediante `(EC)DHE` o una clave precompartida.
4. Con certificados (`Certificate` y `CertificateVerify`) autentican al servidor.
5. `Finished` confirma las claves y la negociación.
6. Comienza el intercambio cifrado y autenticado.

- Un certificado se valida por su cadena de confianza, vigencia, dominio y firmas.

▼ Diferencias entre HTTP vs HTTPS

Característica	HTTP sin TLS	HTTPS
Cifrado	No	Sí
Puerto habitual	80	443
Autenticación del servidor	No	Sí
Uso recomendado	Ejercicios locales	Producción y datos reales

TLS agrega trabajo criptográfico, pero HTTP sin TLS no siempre es más rápido: también influyen reutilización de conexiones y HTTP/2 o HTTP/3.

Webservices - client side

- La biblioteca `requests` permite **enviar solicitudes HTTP** desde Python.
  - Casi todas las solicitudes a servicios externos deben incluir `timeout` para evitar esperas indefinidas.
  - Debemos comprobar el **código de estado** antes de interpretar la respuesta.
  - Para conocer el **formato del cuerpo** se tiene que revisar `Content-Type` en los headers de la respuesta.

▼ Una API suele documentar una **URL base** y distintos **endpoints** para diferentes recursos.

- Por ejemplo, la API **DummyJSON** tiene de base a `https://dummyjson.com/` y uno de sus endpoints es `products/` . De esta forma, si uno quisiera pedir algo a ese endpoint tendría que hacer:
  `respuesta = requests.get(https://dummyjson.com/products/, timeout=20)`

Método `GET`

Obtiene información desde el servidor

`response = requests.get(url, params={key:value}, **kwargs)`

- `url` : **URL** a la que le envías la solicitud.

▼ `params` : diccionario para enviar en la ***query string*** para solicitudes GET

- Se le puede pasar un diccionario con algunos valores que sean listas, así podrías pasar múltiples valores para una misma llave.

- `**kwargs` : headers, cookies, authentication, **timeout**, proxies, verify (SSL), stream, etc.

`requests.get()` retorna un objeto `response`

▼ `response.status_code`

- Retorna el código (número) del estado HTTP

▼ `response.reason`

- Retorna el texto correspondiente al código de estado.
- ej. “Not found” para 404, “OK” para 200, etc.

▼ `response.raise_for_status()`

- Si el código de estado HTTP está entre 200-299, entonces no hace nada porque ese código indica una respuesta correcta.
- Si es **4xx** (error de cliente) o **5xx** (error de servidor), levanta un `HTTPError` con detalles de la solicitud fallida.

▼ `response.json()`

- Retorna un objeto JSON del resultado
  - Si el resultado no fue escrito en formato JSON, entonces levanta un error.

▼ Ejemplo

```
import requests
import json

# Making a get request
response = requests.get('https://httpbin.org/get')

# Decodes JSON data in response_json
response_json = response.json()

# Print json data using for-loop
for key in response_json:
    print(key, ":", response_json[key])

# pretty-print del objeto json
print(json.dumps(response_json, indent=4, sort_keys=True))
```

▼ `response.url`

- Retorna la url de la respuesta.

▼ `response.headers`

- Retorna un objeto que funciona como un **diccionario** que tiene metadata como tipo de contenido, detalles del servidor, cookies, etc.
- Las llaves son *case-insensitive*

Header Name	Description	Example Value
Content-Type	Type of content returned (HTML, JSON, etc.)	application/json; charset=utf-8
Content-Length	Size of the response body in bytes	348
Date	Date and time when the response was generated	Wed, 08 Apr 2025 12:00:00 GMT
Server	Info about the web server	nginx/1.18.0
Set-Cookie	Cookie to be stored on client	sessionId=abc123; Path=/; HttpOnly
Cache-Control	Instructions for caching	no-cache, no-store, must-revalidate
Expires	When the content should be considered outdated	Thu, 01 Jan 1970 00:00:00 GMT
Location	Redirect location	https://example.com/login
Authorization	Info about required authentication (often in requests, too)	Bearer token123
X-RateLimit-Limit	Max number of requests allowed	60

Algunos headers que son frecuentes de encontrar

▼ `response.content`

- Retorna el contenido de la respuesta en bytes.

▼ `response.text`

- Retorna el contenido de la respuesta en unicode.
- Se puede modificar el encoding que usa para esto, ejecutando `response.encoding = nuevo_encoding` antes de llamar a `response.text`

▼ `response.encoding`

- Retorna el encoding que se usa para descifrar el contenido.
- Se puede modificar:
  `response.encoding = nuevo_encoding`

📌 Una solicitud fallida puede retornar un objeto JSON igual, por lo que la entrega de uno de estos no es sinónimo de éxito para la solicitud.
→ Para confirmar que una solicitud se hizo correctamente, usar `response.status_code` o `response.raise_for_status()`

Método `POST`

Envía información al servidor para crear o actualizar recursos

`requests.post(url, params={key:value}, args)`

- Cuando la API espera un JSON, utilizamos el argumento `json=payload` donde `payload` es el diccionario que va a serializar `requests` . De esta forma, `requests` agrega `Content-Type: application/json`
  - ⚠️ el argumento `data=` tiene otra finalidad.
    - Si le pasamos un diccionario, normalmente enviará un **formulario**.

Autenticación mediante headers

- Los headers transmiten metadatos de la solicitud. Uno de sus usos es enviar credenciales, por ejemplo, un token mediante `Authorization` .

▼ Ejemplo de uso con la API DummyJSON

```python
url = "https://dummyjson.com/auth/login"
payload = {
    "username": "emilys",
    "password": "emilyspass",
    "expiresInMins": 30,
}

respuesta = requests.post(url, json=payload, timeout=10)
respuesta.raise_for_status()
TOKEN = respuesta.json()["accessToken"]

print("Token obtenido correctamente; no se muestra por segurida
```

✓ 0.5s Python

```
Token obtenido correctamente; no se muestra por seguridad.
```

```python
url = "https://dummyjson.com/auth/me"
headers = {"Authorization": f"Bearer {TOKEN}"}

respuesta = requests.get(url, headers=headers, timeout=10)
respuesta.raise_for_status()
respuesta.status_code
```

✓ 0.4s Python

```
200
```

📌 **RESUMEN para `requests`**

- `params=` para el query string
- `json=` para cuerpos JSON

- `headers=` para metadatos y credenciales
- `timeout=` para limitar la espera

Webservices - server side

- **WSGI** es la interfaz estándar entre una aplicación web Python y un servidor WSGI.
  - Flask implementa la aplicación; servidores como Waitress o Gunicorn pueden ejecutarla en producción.
  - Los servicios de nube pueden ofrecer infraestructura, plataformas o productos administrados, por lo que no siempre pertenecen a una única categoría.

Servidor con Flask

▼ Crear una app web

```python
from flask import Flask

# Crea objeto de app Flask
app = Flask(__name__)

# .route() le dice a Flask cuál URL debe activar esta función
@app.route('/')
def hello_world():
    return 'Hello World'

if __name__ == '__main__':
    app.run()
```

▼ Endpoints dinámicos

```python
from markupsafe import escape

# <username> permite pasarle como kwarg
# username a la función
app.route('/user/<username>')
def show_user_profile(username):
    # show the user profile for that user
    return f'User {escape(username)}'

# se puede agregar un convertidor para especificar
# el tipo de argumento
app.route('/post/<int:post_id>')
def show_post(post_id):
    # show the post with the given id, the id is an integer
    return f'Post {post_id}'

@app.route('/path/<path:subpath>')
def show_subpath(subpath):
    # show the subpath after /path/
    return f'Subpath {escape(subpath)}'
```

- Tipos de conversores
  - `string` → acepta texto sin slash (/)
  - `int` → acepta enteros positivos

- `float` → acepta decimales positivos
- `path` → acepta texto con slash (/)
- `uuid` → acepta UUID strings

▼ Métodos HTTP

- Por defecto, una ruta solo responde solicitudes `GET` , pero puedes ocupar el argumento `methods` del decorador `route()` para manejar otros métodos HTTP.

```python
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return do_the_login()
    else:
        return show_the_login_form()
```

- También puedes hacerlo con decoradores específicos para cada método HTTP, los cuales vienen con flask

```python
@app.get('/login')
def login_get():
    return show_the_login_form()

@app.post('/login')
def login_post():
    return do_the_login()
```

▼ Argumentos en la URL

- `from flask import request` : `request.args` es un **InmutableMultiDict** con los parámetros del query string.
- `request.args.get(llave, default=, type=)` permite obtener un valor de la query string, considerando un valor por defecto si no se encuentra y también permite indicar un conversor de tipo.

```python
@app_2.route("/argumentos", methods=["GET"])
def argumentos():
    nombre = request.args.get("nombre", default="Guillermo", type=str)
    especie = request.args.get("especie", default="Gato", type=str)
    edad = request.args.get("edad", default=0, type=int)
    return {"texto": f"Hola {especie} {nombre} de {edad} años!"}
```

▼ Datos JSON

- `request.get_json()` interpreta el cuerpo únicamente cuando su tipo de contenido indica JSON, es decir, `Content-Type: application/json`

```python
@app_2.route("/body", methods=["POST"])
def datos_en_body():
    body_data = flask_req.get_json()

    if not isinstance(body_data, dict):
        return {"error": "El cuerpo debe ser un objeto JSON."}, 400
    if not {"var_1", "var_2"} <= body_data.keys():
        return {"error": "Faltan var_1 o var_2."}, 400

    numero_1 = body_data["var_1"]
    numero_2 = body_data["var_2"]
    if not isinstance(numero_1, (int, float)) or not isinstance(
```

```
        numero_2,
        (int, float),
    ):
        return {"error": "Las variables deben ser números."}, 422

    return {
        "var_1": numero_1,
        "var_2": numero_2,
        "result": numero_1 + numero_2,
    }
```

▼ Headers

- `request.headers.get("Authorization")` permite obtener el token de autenticación.

▼ `Response` , `abort`

- `Response(status=200, response="hola", content_type="text/plain; charset=utf-8")`
  - Si el contenido es de tipo json, se puede acceder a este con `Response.get_json()`
    - No confundir con `response.json()` , que hace lo mismo pero para el objeto del módulo `requests`
- `abort(codigo)` detiene la vista mediante una `HTTPException`

▼ `return dict, status_code`

- Flask convierte automáticamente un diccionario a JSON devolviendo código `200 OK` por defecto.
- Se puede especificara un código de error devolviendo una tupla

Race conditions

`ThreadPoolExecutor`

`from concurrent.futures import ThreadPoolExecutor`

- Administra un grupo de hilos reutilizables.

▼ Métodos

- Crear un pool de hasta N hilos
  `ThreadPoolExecutor(max_workers=N)`
- Programa una llamada y retorna un `Future` inmediatamente
  `executor.` **`submit`** `(fn, *args, **kwargs)`
- Retorna un iterador
  `executor.` **`map`** `(fn, iterable)`
  - Las llamadas se ejecutan concurrentemente y los resultados se entregan en orden de entrada.
  - Obtener el siguiente resultado puede bloquear.
- Indica al ejecutor que debe liberar todos los recursos que está utilizando cuando los `Future` actualmente pendientes de ejecución finalicen.
  - `executor.` **`shutdown`**
- Espera el resultado o vuelve a levantar la excepción
  `future.` **`result`** `()`
- Entrega los `Future` a medida que terminan
  `as_completed` `(futures)`
  - Retorna un iterador de las instancias de `Future` (posiblemente creadas por distintas instancias de `Executor`) que se le pasan con `futures`

▼ Ejemplo

```python
import concurrent.futures
import urllib.request

URLS = ['http://www.foxnews.com/',
        'http://www.cnn.com/',
        'http://europe.wsj.com/',
        'http://www.bbc.co.uk/',
        'http://nonexistent-subdomain.python.org/']

# Retrieve a single page and report the URL and contents
def load_url(url, timeout):
    with urllib.request.urlopen(url, timeout=timeout) as conn:
        return conn.read()

# We can use a with statement to ensure threads are cleaned up promptly
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    # Start the load operations and mark each future with its URL
    future_to_url = {executor.submit(load_url, url, 60): url for url in URLS}
    for future in concurrent.futures.as_completed(future_to_url):
        url = future_to_url[future]
        try:
            data = future.result()
        except Exception as exc:
            print('%r generated an exception: %s' % (url, exc))
        else:
            print('%r page is %d bytes' % (url, len(data)))
```

Al salir de `with` se ejecuta `shutdown(wait=True)` , por lo que se espera a los trabajos pendientes.

📌

▼ Una **condición de carrera** ocurre cuando el resultado depende un orden de ejecución no coordinado entre operaciones concurrentes.

```
Hilo A: lee 5
Hilo B: lee 5
Hilo A: escribe 6
Hilo B: escribe 6  ← se perdió una actualización
```

Este caso se denomina *lost update*. Las condiciones de carrera pueden afectar memoria compartida, archivos, bases de datos y recursos modificados mediante APIs.

Serialización con `pickle`

- El módulo `pickle` permite guardar y cargar casi **cualquier objeto de Python**.
  - `dumps` serializa un objeto, es decir, lo **guarda** como una secuencia de **bytes**.
  - `loads` deserializa un objeto, es decir, **carga** un objeto a su estado **original**.

    ▼ Esto funciona incluso con clases personalizadas.

    - Cuando uno hace `objeto_bytes = pickle.dumps(instancia)` , se guardan los atributos de la instancia, pero también se obtiene el nombre y módulo de la clase. Así, cuando uno llama a `pickle.loads(objeto_bytes)` , obtiene directamente la instancia de clase original.

  - `dump` guarda un archivo con el objeto serializado
  - `load` deserializa un objeto almacenado en un archivo

- Una vez que un objeto es serializado, este es **persistente** y está listo para ser reutilizado en el futuro.
- Como `pickle` funciona con bytes, cuando trabajamos con archivos debemos hacerlo en modo binario (ej. `"wb"` , `"rb"` , etc.)

> 📌 **Nunca debes carga un archivo *pickle* cuando no conoces su procedencia**

Personalizar la (de)serialización

Serialización

- `pickle` revisa el método `__getstate__` del objeto.
  - Si no está implementado, entonces va a ocupar el atributo `__dict__` del objeto.

▼ Ejemplo de implementación de `__getstate__`

```python
class Persona:

    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre
        self.edad = edad
        self.mensaje = "No pasa nada"

    def __getstate__(self) -> dict:
        """
        Retorna el estado actual del objeto, para que sea serializado por pickle

        Aquí creamos una copia del diccionario actual, para modificar la copia
        y no el objeto original
        """
        # Usamos una copia del diccionario original para alterar solo la copia
        nueva = self.__dict__.copy()
        # Modificamos un atributo en el objeto serializado.
        # Sin embargo, el objeto original no ha cambiado.
        nueva.update({"mensaje": "¡Me están serializando!"})
        # Lo que retornemos es lo que será serializado por pickle
        return nueva
```

Deserialización

- `pickle` revisa el método `__setstate__` del objeto.
  - Recibe como argumento el diccionario que representa el estado del objeto que fue serializado.
  - Debe asignarlo al diccionario del objeto (`__dict__`)

▼ Ejemplo de implementación

```python
class Persona:

    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre
        self.edad = edad
        self.mensaje = "No pasa nada"

    def __getstate__(self) -> dict:
        nueva = self.__dict__.copy()
        print(f"[__getstate__] Serializando a {nueva['nombre']}")
        nueva.update({"mensaje": "¡Me están serializando!"})
        # Lo que retornemos es lo que será serializado por pickle
        return nueva
```

```
    def __setstate__(self, state) -> None:
        print("[__setstate__] Objeto recién deserializado, actualizando su estad
o")
        # Al desarializar modificamos el estado
        state.update({"nombre": f"{state['nombre']} deserializado"})
        self.__dict__ = state
```

- Si no está implementado `__setstate__`, entonces el estado deserializado se asignará al `__dict__` del objeto sin realizar otras acciones adicionales.

📌 Una aplicación de la personalización de (de)serialización es cuando necesitamos serializar un objeto que contiene un **atributo que depende de las condiciones actuales** del programa.

Manejo de bytes

▼ Friendly reminder

- Byte = 8 bits → almacena $2^8 = 256$ combinaciones diferentes de 0s y 1s
- 0xAA, 0x1E, 0x23 → números hexadecimales
  - ej. 256 (decimal) es 0x01/0x00
- La tabla ASCII abarca esas 256 combinaciones, por lo que se queda corta ante símbolos de puntuación u otros idiomas.

- `chr(int)` retorna el caracter asociado a `int` (valor **decimal**)
- `ord(char)` retorna el código decimal asociado a `char`
- `hex(int)` retorna la representación hexadecimal de un número entero

📌
- La representación hexadecimal de un **byte** utiliza dos dígitos hexadecimales para representar los 8 bits.
- **Unicode** es el encoding principal para el curso

- ⚠️ No confundir **0x** con **/x**

  ▼ **0x → representación hexadecimal**

  - ej. 0x00 es el número 0

  ▼ **/x → l**os siguientes 2 caracteres después de la x son un **byte** usando dígitos hexadecimales

  - ej. /x01/x00 es el número 256 (si está en bigendian)

Objeto `bytes`

- Un objeto de tipo `bytes` es **inmutable**.

▼ Para crear un objeto `bytes` se pone una `b` al lado del string

```
caracteres = b"\x63\x6x\x69\x63\x68\xe9"
palabras = b"hola mundo"
errror = b"'ab" # solo acepta literales ASCII
```

▼ Los bytes pueden representar cualquier cosa. Para poder fijar cómo se deben interpretar se debe entregar la **codificación** que decodificarlos

```
caracteres = b'\x63\x6c\x69\x63\x68\xe9'
print(caracteres.decode("latin-1"))
print(caracteres.decode("iso8859-5"))
print(caracteres.decode("CP437"))
print(caracteres.decode("utf-16"))
```

▼ Para codificar un `string` en un alfabeto específico, simplemente se usa el método `str.encode()`

```
caracteres = "estación"
print(caracteres.encode("UTF-8"))  # 8-bit Unicode Transformation Format
print(caracteres.encode("latin-1"))
print(caracteres.encode("CP437"))

# No se puede codificar en ASCII el caracter "ó" ya que no existe dentro
# de los 128 caracteres de ASCII
print(caracteres.encode("ascii"))

# Encode permite especificar cómo manejar errores
print(caracteres.encode("ascii", errors='replace'))  # en ascii se reemplaza el car
acter desconocido con "?"
print(caracteres.encode("ascii", errors='ignore'))
print(caracteres.encode("ascii", errors='xmlcharrefreplace'))  # se crea una entida
d xml que representa el caracter Unicode
```

📌 En general, la mejor opción es usar **UTF-8,** ya que es *backwards* compatible con ASCII (coinciden en los primeros 128 caracteres)

Objeto `bytearray`

- Arreglos (listas) de `bytes` que son **mutables.**

▼ Se crean con la función `bytearray()` que recibe un `byte` como argumento

```
ba = bytearray(b"holamundo")
print(ba)

# Podemos ocupar la notación de slicing
print(ba[3:7])
ba[4:6] = b"\x15\xa3"
print(ba)

# Podemos agregar bytes con el método extend
ba.extend(b"programa")
print(ba)
```

- Permiten realizar la mayoría de operaciones de las listas
  - Slicing

▼ `bytearray.extend()`

```
mi_bytearray = bytearray()

# El método extend funciona con byte o bytearray
mi_bytearray.extend(b'\xff') #extendemos con un byte
```

```
print(mi_bytearray)

mi_bytearray.extend(bytearray(b'\xff\x12')) #extendemos con otro bytearray
print(mi_bytearray)
```

▼ Indexación

```
# Aquí se imprime un byte, representado por un entero, el ascii que corresponde
a la letra "h"
print(ba[0])
# La función bin genera un string con la representación binaria del byte
print(bin(ba[0]))
# El método zfill(r) rellena con 0's al inicio hasta completar r caracteres
print(bin(ba[0])[2:].zfill(8))

# Recordemos que imprimir un elemento de un bytearray nos muestra el int que le
representa
print(bytearray(b"a")[0])
# La función ord hace esta conversión directamente a partir de un byte
print(ord(b"a"))

# Aqui ocupamos ambas formas (int directamente y ord) para hacer cambios en un b
ytearray
b = bytearray(b'abcdef')
b[3] = ord(b'g')         # El caracter g tiene como código ascii el 103
b[4] = 68                # El caracter D tiene como código ascii el 68, esto ser
ía lo mismo que ingresar b[4] = ord(b'D')
print(b)
```

▼ `bytearray.` **`append()`**

```
mi_bytearray = bytearray()

# El método append solo funciona en bytearray con un int
mi_bytearray.append(2)
print(mi_bytearray)
```

- **`max`** `(bytearray)`
- `for x in bytearray:`

Chunks

- Un **chunk** es un grupo de bytes.

▼ Podemos imprimir un bytearray por chunks, usando slicing

```
muchos_bytes = bytearray(b"Una gran cantidad de bytes que quiero leer de a poquit
o")
TAMANO_CHUNK = 4

for i in range(0, len(muchos_bytes), TAMANO_CHUNK):
    # Aqui obtenemos nuestro chunk
    chunk = bytearray(muchos_bytes[i:i+TAMANO_CHUNK])
    print(chunk)
```

▼ Podemos leer archivos por chunks, usando el modo de lectura `"rb"` y `file.read(n)`

```python
from os.path import join

TAMANO_CHUNK = 4
with open(join('data', 'ejemplo_chunk.bin'), 'rb') as file:
    while chunk := file.read(TAMANO_CHUNK):
        print(chunk)
```

`:=` se llama **walrus operator.** Permite definir la variable a su izquierda con el valor de la expresión de la derecha. Si lo que se lee está vacío (se ha llegado al final del documento), entonces `chunk` queda como vacío y se detiene el `while`

Transformar números

- 1 byte permite representar un número entero entre 0 y 255, incluidos.
- Para representar números más grandes se ocupan múltiples bytes (un **bytearray**) que se escriben en un **orden** en particular, el cual puede ser de 2 tipos:
  - **big endian** → el byte más significativo (de mayor peso) queda al inicio del byte array

    ▼ Se parece a como escribimos números en lenguaje natural

    - ej. 1242 = 1*1000 + 2*100 + 4*10 + 2*1
    - ej. 269 = \x01\x0d = (1)*256 + (13)*256^0
  - **little endian** → el byte más significativo (de mayor peso) queda al final del byte array.
- Conversiones `int` ← → `byte`

  ▼ `int.` **`to_bytes`** `(lenght=1, byteorder="big")`

  - Retorna un bytearray.
  - Ambos argumentos son obligatorios.

  ▼ Si `length` es más grande que la cantidad de bytes necesaria para representar el número, la función rellena con ceros.

  - ej con big: 5 → 005

  - `byteorder` puede ser `"big"` (big endian) o `"little"` (little endian)

  ▼ `int.` **`from_bytes(`** `byte, byteorder="big")`

  - Retorna un número.
  - Si el byte es de largo uno, el `byteorder` no influye

Print de bytes

- Cuando se hace `print` , el compu puede reemplazar visualmente un byte por otro carácter.
- Se recomienda **evitar comprobar operaciones con print** y, en su lugar, ocupar métodos alternativos (ej. convertir bytes a int y comparar números).

Networking con python

Sockets

- Objeto del sistema operativo que permite la comunicación entre programas distintos.
- El módulo **`socket`** permite crear un socket como una instancia de la clase **`socket`** `(family,type)` donde

  ▼ `family` → tipo de dirección IP

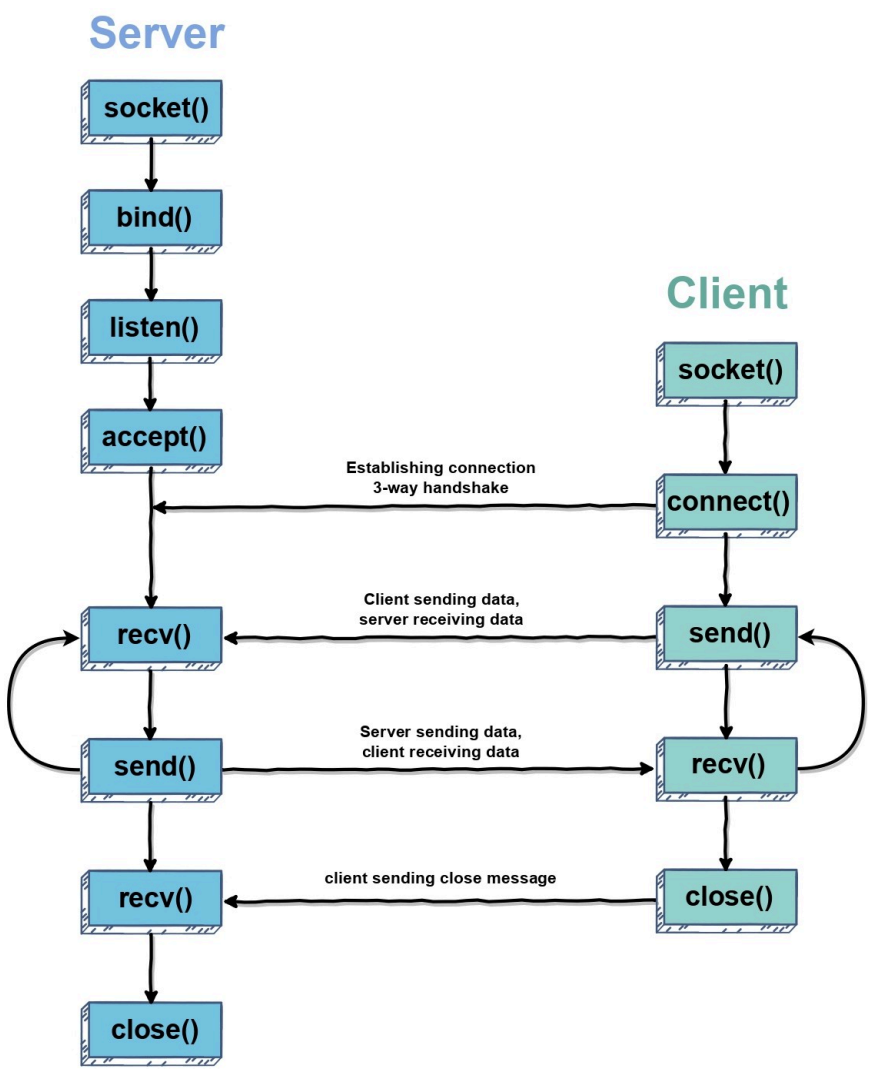
  - `socket.AF_INET` para IPv4

- `socket.AF_INET6` para IPv6

▼ `type` → protocolo de transporte

- `socket.SOCK_STREAM` para TCP
- `socket.SOCK_DGRAM` para UDP

Esquema de sockets (TCP)



📌

- La `address` a utilizar con los sockets depende de la **familia de IPs** con la que fue configurado. En particular, consisten en tuplas:
  - IPv4 → `(host, port)`
  - IPv6 → `(host, port, flowinfo, scope_id)`

Cliente TCP

1. Crear una instancia de socket → `ejemplo = socket(family, socket.SOCK_STREAM)`
   - Debe tener la misma configuración que el socket del servidor de destino
2. Conectarse al servidor → `ejemplo.connect(address)`
   - `host` : dirección IP del destinatario o hostname (ej. `github.com`)
   - `port` : puerto del host destinatario
3. Enviar mensajes codificados → `send()` o `sendall()`
   - Los sockets sólo transmiten **bytes**

▼ `ejemplo.send(bytes)`

- Trata de enviar el mensaje completo, pero podría llegar a enviar sólo una parte de este o que el envío resulte en error.
- Retorna la cantidad de bytes que fue enviada efectivamente → **uno debe programar los envíos sucesivos** hasta que la cantidad enviada total sea igual al largo del mensaje original.

▼ `ejemplo.sendall(bytes)`

- Envía el mensaje completo o retorna un error en caso de no ser posible.
- No permite ver cuántos bytes lograron enviarse.

4. Recibir respuestas codificadas → `recv()`

▼ `ejemplo.recv(buffer)`

- Retorna un objeto `bytes`
- `buffer` : cantidad máxima de bytes que será leída
- Se sugiere ocupar una potencia de 2 pequeña, como 4096.
- Es **bloqueante** de thread, o sea, se detiene hasta recibir algo.

5. Cerrar la conexión → `ejemplo.close()`
   - Evita malgasto de recursos.

📌 La clase `ConnectionError` es la clase base para errores relacionados con problemas de conexión.

▼ Ejemplo de Cliente con socket TCP

```python
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

host = "stackoverflow.com"
port = 80

try:
    sock.connect((host, port))
    sock.sendall('GET / HTTP/1.1\nHost: stackoverflow.com\n\n'.encode('utf-8'))
    data = sock.recv(4096)
    print(data.decode('utf-8'))
except ConnectionError as e:
    # ConnectionError es la clase base de BrokenPipeError, ConnectionAbortedError,
    # ConnectionRefusedError y ConnectionResetError
    print("Ocurrió un error.")
finally:
    # ¡No olvidemos cerrar la conexión!
    sock.close()
```

📌 **Observación**

- El puerto 80 es usado por servidores de texto plano, esto es, sin encriptación.
- → **HTTPS** funciona en el puerto **443**.

Servidor TCP

1. Iniciar socket → `ejemplo = socket.socket(family, socket.SOCK_STREAM)`
2. Enlazar socket a un puerto → `ejemplo.bind(address)`
   - Se ocupa `host="localhost"` para correrlo local.
3. Escuchar conexiones → `ejemplo.listen(n)`
   - `n` : indica cuántas conexiones pendientes (sin aceptar) pueden estar de forma paralela. Así, la conexión `n+1` va a ser rechazada.
4. Aceptar conexiones → `ejemplo.accept()`

- Retorna una tupla `(conn, address)` donde `conn` es un objeto socket para comunicarse con el cliente y `address` es la dirección del cliente
- Es una llamada **bloqueante**, es decir, el programa se detiene hasta que un cliente se intenta conectar.

▼ Ejemplo de uso

```
while True:
    socket_cliente, address = sock.accept()
    print("Conexión aceptada desde", address)
    socket_cliente.sendall("Gracias por
conectarte\n".encode("utf-8"))
    socket_cliente.close()
```

Acepta clientes por siempre

```
counter = 0
while counter < 5:
    try:
        socket_cliente, address = sock.accept()
        print("[Servidor] Conexión aceptada desde",
address)
        socket_cliente.sendall("Gracias por
conectarte\n".encode("utf-8"))
        socket_cliente.close()
        counter += 1
    except ConnectionError:
        print("[Servidor] Ocurrió un error.")
```

Acepta exactamente 5 clientes

5. Terminar el programa → `ejemplo.close()`

📌 A nivel del cliente, cuando usamos `socket.connect()` , el SO se encarga de asignarle un **puerto aleatorio** al cliente. Sin embargo, si el servidor exigiera un rango específico de puertos, ahí sí tendríamos que "enlazar" el cliente a un puerto específico también.

▼ No confundir el host y puerto en connect() con respecto a los de bind()

- connect() se usa en el cliente → recibe host, puerto del servidor
  - El SO se encarga de asignarle un puerto de **salida** al cliente.
- bind() se usa en el servidor → recibe host, puerto del servidor

Ejemplos de servidores con socket TCP

▼ Servidor echo + cliente envía JSONs (codificados)

```python
# Implementación del servidor que recibe datos y los envía de vuelta.
# Esto comúnmente se denomina como 'echo server'.

import socket


host = 'localhost'   # Usar nuestro computador como host
port = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind((host, port))
print("Listo para recibir. Ahora voy a esperar que me hablen...")
sock.listen()

sock_cliente, (host_cliente, puerto_cliente) = sock.accept()
print("Conexión desde", host_cliente, puerto_cliente)

while True:
    data = sock_cliente.recv(4096)
    print(f"Recibí estos bytes: {data}")
    if not data:
        break
    print("Los enviaré de vuelta")
    sock_cliente.sendall(data)

sock_cliente.close()
sock.close()
```

Configuración del servidor. Podemos recibir información desde el cliente ocupando su socket (`sock_cliente`) y el método adecuado (`recv()`)

```python
# Implementación del cliente que envía los datos en formato JSON.
# Pon atención en la serialización y transformación a bytes.

import json
import socket


server_host = ''  # Debemos poner aquí la dirección IP del servidor.
# Si no ponemos nada, supone que estamos hablando con un programa en el mismo host.
server_port = 12345

# Generamos la información que enviaremos; en este caso, es un simple diccionario.
data = {1: "Hola", 2: "Chao"}
mensaje = json.dumps(data)

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((server_host, server_port))

# Debemos enviar bytes, por lo que codificamos el mensaje.
sock.sendall(mensaje.encode('utf-8'))

# Decodificamos los bytes de la respuesta y luego los deserializamos con JSON.
data = json.loads(sock.recv(4096).decode('utf-8'))
print(data)
input("ENTER para terminar.")
sock.close()
```

Configuración del cliente. Para enviar y recibir JSON debe realizar la codificación y decodificación correspondientes, junto con la serialización y deserialización.

▼ Servidor + cliente para transferencia de archivos de gran tamaño

- Puntos clave
  - El cliente lee el archivo y agrega el tamaño de este en los primeros bytes que envía al servidor.
    - Si pasamos un identificador de 4 bytes, esto significa que podemos representar hasta 2^{32} bits (~ 4 Gb).
  - El servidor recibe el tamaño del archivo y se organiza para recibir por chunks hasta completar el tamaño del archivo.
    - En lugar de `socket.sendall()`, utiliza `socket.send()` con un loop para asegurarse de enviar la totalidad del documento
  - Para evitar leer bytes de un mensaje posterior, ajusta los bytes a leer según lo que falta por recibir:
    - `bytes_a_leer = min(4096, total - len(datos))`
  - `socket.recv()` no retorna la cantidad de bytes que recibió → ocupar `len(datos)` donde `datos=socket.recv()`

▼ Implementación completa

```python
# Emisor mejorado del archivo.
import socket


host_receptor = 'localhost'
puerto_receptor = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Nos conectamos al receptor del archivo, que ya debería estar escuchando.
sock.connect((host_receptor, puerto_receptor))
print("Conexión establecida.")

# Leemos el archivo y lo enviamos.
with open('files/enviar.bin', 'rb') as binfile:
    datos = binfile.read()
    largo_archivo = len(datos)
    print(f"Voy a enviar {largo_archivo} bytes")
    # 1. Enviar el tamaño del archivo.
    # (int.to_bytes transforma un entero en una cantidad de bytes por el primer parámetro)
    sock.sendall(largo_archivo.to_bytes(4, byteorder='big'))

    # 2. Enviar el archivo. Para esto haremos uso del método send en vez de sendall, para asegurarnos
    # que todo se envíe correctamente
    bytes_enviados = 0
    while bytes_enviados < largo_archivo:
        # Enviaremos el mensaje cortado desde el último punto efectivamente enviado.
        bytes_enviados += sock.send(datos[bytes_enviados:])


print("¡Archivo enviado!")

# Imprimimos lo que nos responda la contraparte.
print("Respuesta:", sock.recv(4096).decode('utf-8'))

# Cerramos el socket.
sock.close()
```

Configuración del cliente

```python
# Receptor mejorado del archivo.
import socket
host = 'localhost'
puerto = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind((host, puerto))
sock.listen()
print('Escuchando...')

# Aceptamos un cliente.
sock_cliente, (host_cliente, puerto_cliente) = sock.accept()
print("Conexión entrante aceptada.")

# Leemos primero los 4 bytes del tamaño del archivo.
# Con esto transformamos una serie de bytes en un int.
largo_archivo = int.from_bytes(sock_cliente.recv(4), byteorder='big')
datos = bytearray()
bytes_leidos = 0
print(f"OK. Ahora sé que debo recibir {largo_archivo} bytes")

# Ahora leemos el archivo por chunks, de máximo 4096 bytes.
while len(datos) < largo_archivo:
    # El último recv será probablemente más chico que 4096
    bytes_leer = min(4096, largo_archivo - len(datos))
    datos_recibidos = sock_cliente.recv(bytes_leer)
    # Recordemos que el método recv, entrega una cantidad máxima, pero no nos asegura que nos
    # entregue los 4096 bytes. Es por esto, que la cantidad de bytes que hemos recibido en
    # total, se deben ver siempre en función de lo que retornó el método recv, y no lo que
    # le entregamos como parámetro
    bytes_leidos += len(datos_recibidos)
    print(
        f"He recibido {len(datos_recibidos)} bytes en el último recv. Van {bytes_leidos} en total.")
    datos.extend(datos_recibidos)

print(f"¡Listo! He recibido {len(datos)} bytes")

# Guardamos la información en un archivo.
with open('files/recibido.bin', 'wb') as binfile:
    binfile.write(datos)

print("¡Archivo guardado!")
# Le enviamos una respuesta a la contraparte.
sock_cliente.sendall("Gracias.".encode('utf-8'))

# Cerramos los sockets.
sock_cliente.close()
sock.close()
```

Configuración del servidor

▼ Servidor con manejo concurrente de múltiples clientes vía threads

+ cliente con thread específico para servidor

- Puntos clave
  - Servidor

    ▼ (1) Tiene un thread enfocado exclusivamente a aceptar las conexiones y generar un thread individual para cada una de estas.

```
    # métodos de class Servidor:

    def accept_connections(self) -> None:
        """
        Inicia el thread que aceptará clientes.

        Aunque podríamos aceptar clientes en el thread principal de la
        instancia, es útil hacerlo en un thread aparte. Esto nos
        permitirá realizar la lógica en la parte del servidor sin dejar
        de aceptar clientes. Por ejemplo, seguir procesando archivos.
        """
        thread = threading.Thread(target=self.accept_connections_thread)
        thread.start()

    def accept_connections_thread(self) -> None:
        """
        Es arrancado como thread para aceptar clientes.

        Cada vez que aceptamos un nuevo cliente, iniciamos un
        thread nuevo encargado de manejar el socket para ese cliente.
        """
        print("Servidor aceptando conexiones...")

        while True:
            client_socket, _ = self.socket_server.accept()
            listening_client_thread = threading.Thread(
                target=self.listen_client_thread,
                args=(client_socket, ),
                daemon=True)
            listening_client_thread.start()
```

`listen_client_thread` es el método que implementemos para manejar la interacción con un cliente

▼ (2) La función del thread de cada cliente se encarga de implementar la funcionalidad del servidor

```
    # método de class Servidor:

    def listen_client_thread(self, client_socket: socket.socket) -> None:
        """
        Es ejecutado como thread que escuchará a un cliente en particular.

        Implementa las funcionalidades del protocolo de comunicación
        que permiten recuperar la informacion enviada.
        """
        print(f"{client_socket.getpeername()} Nuevo cliente conectado al se
rvidor")

        while True:
            response_bytes_length = client_socket.recv(4)
            response_length = int.from_bytes(
                response_bytes_length, byteorder='big')
            response = bytearray()

            while len(response) < response_length:
                read_length = min(4096, response_length - len(response))
                response.extend(client_socket.recv(read_length))

            received = response.decode()

            if received != "":
                # El método `self.handle_command()` debe ser definido.
                # Este realizará toda la lógica asociado a los mensajes
                # que llegan al servidor desde un cliente en particular.
                # Se espera que retorne la respuesta que el servidor
                # debe enviar hacia el cliente.
                response = self.handle_command(received, client_socket)
                self.send(response, client_socket)
```

`socket.` `getpeername()` : retorna la address remota a la que está conectado el socket. `handle_command()` : función que implementamos para realizar lo que necesitemos con los mensajes y retornar la respuesta para el cliente. `send()` : función que implementamos para seguir el protocolo de la aplicación.

- Cliente

▼ (1) Establece un thread dedicado para estar recibiendo los mensajes desde el servidor

```
# métodos de class Cliente:
 def listen(self) -> None:
      """
      Inicializa el thread que escuchará los mensajes del servidor.

      Es útil hacer un thread diferente para escuchar al servidor,
      ya que de esa forma podremos tener comunicación asíncrona con este.
      Luego, el servidor nos podrá enviar mensajes sin necesidad de
      iniciar una solicitud desde el lado del cliente.
      """

      thread = threading.Thread(target=self.listen_thread, daemon=True)
      thread.start()

  def listen_thread(self) -> None:
      while True:
          response_bytes_length = self.socket_client.recv(4)
          response_length = int.from_bytes(
              response_bytes_length, byteorder='big')
          response = bytearray()

          # Recibimos datos hasta que alcancemos la totalidad de los dato
s
          # indicados en los primeros 4 bytes recibidos.
          while len(response) < response_length:
              read_length = min(4096, response_length - len(response))
              response.extend(self.socket_client.recv(read_length))

          print(f"{response.decode()}\n>>> ", end='')
```

En este caso solo se dedica a imprimir la respuesta del servidor.

- Servidor y Cliente

▼ Ambos implementan el mismo protocolo de aplicación para enviarse mensajes. En concreto, calculan el tamaño del mensaje y lo agregan como los primeros 4 bytes de este.

```
    def send(self, msg: str) -> None:
        """
        Envía mensajes al servidor.

        Implementa el mismo protocolo de comunicación que mencionamos;
        es decir, agregar 4 bytes al principio de cada mensaje
        indicando el largo del mensaje enviado.
        Luego, puedes elegir entre usar send() en un ciclo, o un solo senda
ll().
        En este ejemplo, solo usamos un sendall(), pero puedes cambiarlo po
r
        el ciclo del ejemplo anterior para asegurar que toda la información
        se enviará correctamente.
        """

        msg_bytes = msg.encode()
        msg_length = len(msg_bytes).to_bytes(4, byteorder='big')
        self.socket_client.sendall(msg_length + msg_bytes)
```

Como indica el comentario, esta configuración **no asegura el envío del mensaje.** En caso de necesitar asegurarlo, debería ocupar un loop con `socket.send()` como en el ejemplo anterior

▼ Implementación completa

```python
import socket
import threading


class Server:
    def __init__(self, port: int, host: str) -> None:
        print("Inicializando servidor...")

        self.host = host
        self.port = port
        self.socket_server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.bind_and_listen()
        self.accept_connections()

    def bind_and_listen(self) -> None:
        """
        Enlaza el socket creado con el host y puerto indicado.

        Primero, se enlaza el socket y luego queda esperando
        por conexiones entrantes.
        """
        self.socket_server.bind((self.host, self.port))
        self.socket_server.listen()
        print(f"Servidor escuchando en {self.host}:{self.port}...")

    def accept_connections(self) -> None:
        """
        Inicia el thread que aceptará clientes.

        Aunque podríamos aceptar clientes en el thread principal de la
        instancia, es útil hacerlo en un thread aparte. Esto nos
        permitirá realizar la lógica en la parte del servidor sin dejar
        de aceptar clientes. Por ejemplo, seguir procesando archivos.
        """
        thread = threading.Thread(target=self.accept_connections_thread)
        thread.start()

    def accept_connections_thread(self) -> None:
        """
        Es arrancado como thread para aceptar clientes.

        Cada vez que aceptamos un nuevo cliente, iniciamos un
        thread nuevo encargado de manejar el socket para ese cliente.
        """
        print("Servidor aceptando conexiones...")

        while True:
            client_socket, _ = self.socket_server.accept()
            listening_client_thread = threading.Thread(
                target=self.listen_client_thread,
                args=(client_socket, ),
                daemon=True)
            listening_client_thread.start()

    @staticmethod
    def send(value: any, sock: socket.socket) -> None:
        """
        Envía mensajes hacia algún socket cliente.

        Debemos implementar en este método el protocolo de comunicación
        donde los primeros 4 bytes indicarán el largo del mensaje.
        Luego, puedes elegir entre usar send() en un ciclo, o un solo sendall().
        En este ejemplo, solo usamos un sendall(), pero puedes cambiarlo por
        el ciclo del ejemplo anterior para asegurar que toda la información
        se enviará correctamente.
        """
        stringified_value = str(value)
        msg_bytes = stringified_value.encode()
        msg_length = len(msg_bytes).to_bytes(4, byteorder='big')
        sock.sendall(msg_length + msg_bytes)

    def listen_client_thread(self, client_socket: socket.socket) -> None:
        """
        Es ejecutado como thread que escuchará a un cliente en particular.

        Implementa las funcionalidades del protocolo de comunicación
        que permiten recuperar la información enviada.
        """
        print(f"{client_socket.getpeername()} Nuevo cliente conectado al servidor")

        while True:
            response_bytes_length = client_socket.recv(4)
            response_length = int.from_bytes(
                response_bytes_length, byteorder='big')
            response = bytearray()

            while len(response) < response_length:
                read_length = min(4096, response_length - len(response))
                response.extend(client_socket.recv(read_length))

            received = response.decode()

            if received != "":
                # El método `self.handle_command()` debe ser definido.
                # Este realizará toda la lógica asociado a los mensajes
                # que llegan al servidor desde un cliente en particular.
                # Se espera que retorne la respuesta que el servidor
                # debe enviar hacia el cliente.
                response = self.handle_command(received, client_socket)
                self.send(response, client_socket)

    def handle_command(self, received: str, client_socket: socket.socket) -> str:
        print(f"{client_socket.getpeername()} Comando recibido:", received)
        # Este método debería ejecutar la acción y enviar una respuesta.
        return "Acción asociada a " + received


if __name__ == "__main__":
    port = 8080
    host = 'localhost'

    server = Server(port, host)
```

Configuración del servidor

```python
import socket
import threading


class Client:
    """
    Maneja toda la comunicación desde el lado del cliente.

    Implementa el esquema de comunicación donde los primeros 4 bytes de cada
    mensaje indicarán el largo del mensaje enviado.
    """

    def __init__(self, port: int, host: str) -> None:
        print("Inicializando cliente...")

        self.host = host
        self.port = port
        self.socket_client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        try:
            self.connect_to_server()
            self.listen()
            self.repl()
        except ConnectionError:
            print("Conexión terminada.")
            self.socket_client.close()
            exit()

    def connect_to_server(self) -> None:
        """Crea la conexión al servidor."""

        self.socket_client.connect((self.host, self.port))
        print("Cliente conectado exitosamente al servidor.")

    def listen(self) -> None:
        """
        Inicializa el thread que escuchará los mensajes del servidor.

        Es útil hacer un thread diferente para escuchar al servidor,
        ya que de esa forma podremos tener comunicación asíncrona con este.
        Luego, el servidor nos podrá enviar mensajes sin necesidad de
        iniciar una solicitud desde el lado del cliente.
        """

        thread = threading.Thread(target=self.listen_thread, daemon=True)
        thread.start()

    def send(self, msg: str) -> None:
        """
        Envía mensajes al servidor.

        Implementa el mismo protocolo de comunicación que mencionamos;
        es decir, agregar 4 bytes al principio de cada mensaje
        indicando el largo del mensaje enviado.
        Luego, puedes elegir entre usar send() en un ciclo, o un solo sendall().
        En este ejemplo, solo usamos un sendall(), pero puedes cambiarlo por
        el ciclo del ejemplo anterior para asegurar que toda la información
        se enviará correctamente.
        """

        msg_bytes = msg.encode()
        msg_length = len(msg_bytes).to_bytes(4, byteorder='big')
        self.socket_client.sendall(msg_length + msg_bytes)

    def listen_thread(self) -> None:
        while True:
            response_bytes_length = self.socket_client.recv(4)
            response_length = int.from_bytes(
                response_bytes_length, byteorder='big')
            response = bytearray()

            # Recibimos datos hasta que alcancemos la totalidad de los datos
            # indicados en los primeros 4 bytes recibidos.
            while len(response) < response_length:
                read_length = min(4096, response_length - len(response))
                response.extend(self.socket_client.recv(read_length))

            print(f"{response.decode()}\n>>> ", end='')

    def repl(self) -> None:
        """
        Captura el input del usuario.

        Lee mensajes desde el terminal y después se pasan a `self.send()`.
        """

        print("------ Consola ------\n>>> ", end='')
        while True:
            msg = input()
            self.send(msg)


if __name__ == "__main__":
    port = 8080
    host = 'localhost'

    client = Client(port, host)
```

Configuración del cliente

▼ Cliente conectado a múltiples servidores

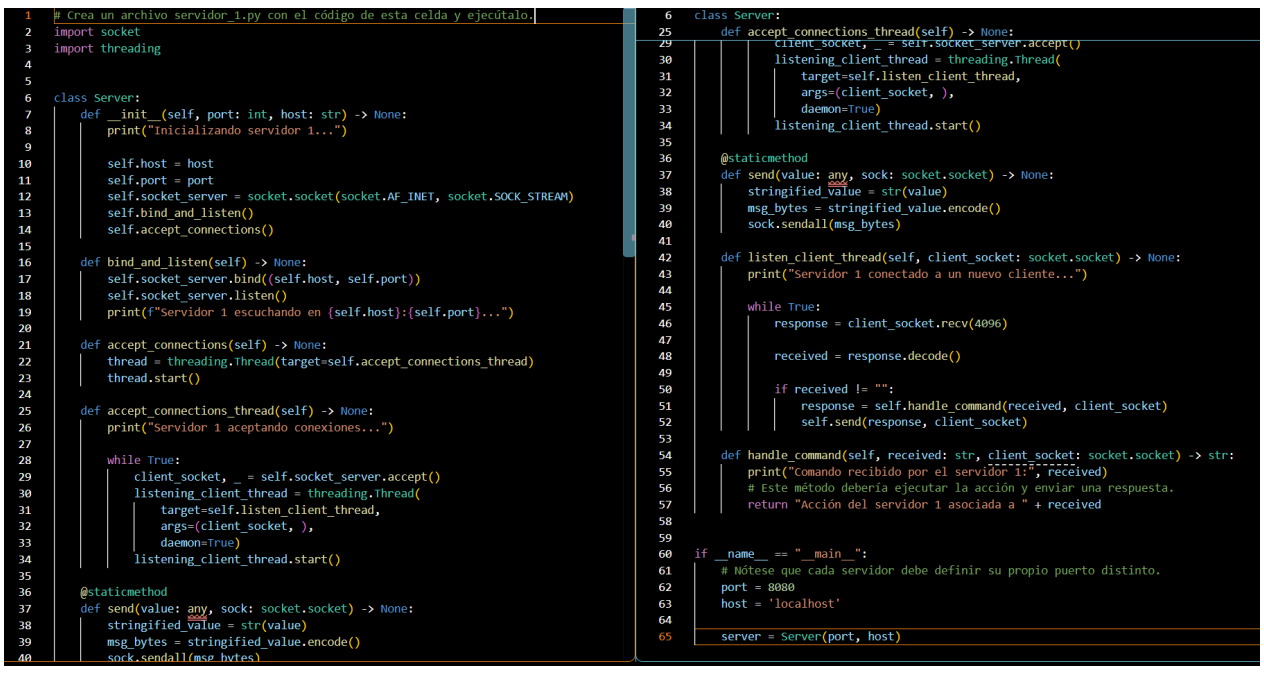
- Puntos clave
  - Cliente recibe un diccionario con la información de cada servidor. Luego, crea un socket para cada uno y los deja en un atributo para poder acceder a estos.
  - Para cada servidor, crea un thread que se encargue de recibir mensajes desde ese servidor. En particular, a cada thread le pasa una función general para escuchar a los servidores, cuya única distinción (con respecto a previos ejemplos) es acceder al socket a través del atributo de la clase Cliente.
  - La función `reply` es una implementación **simple** para enviar mensajes a múltiples servidores.
    - Permite que el usuario seleccione un servidor, pero está constantemente preguntando esto.

▼ 💡 Tal vez podría implementarse queues (del módulo `queue`) para tener **threads especializados por**

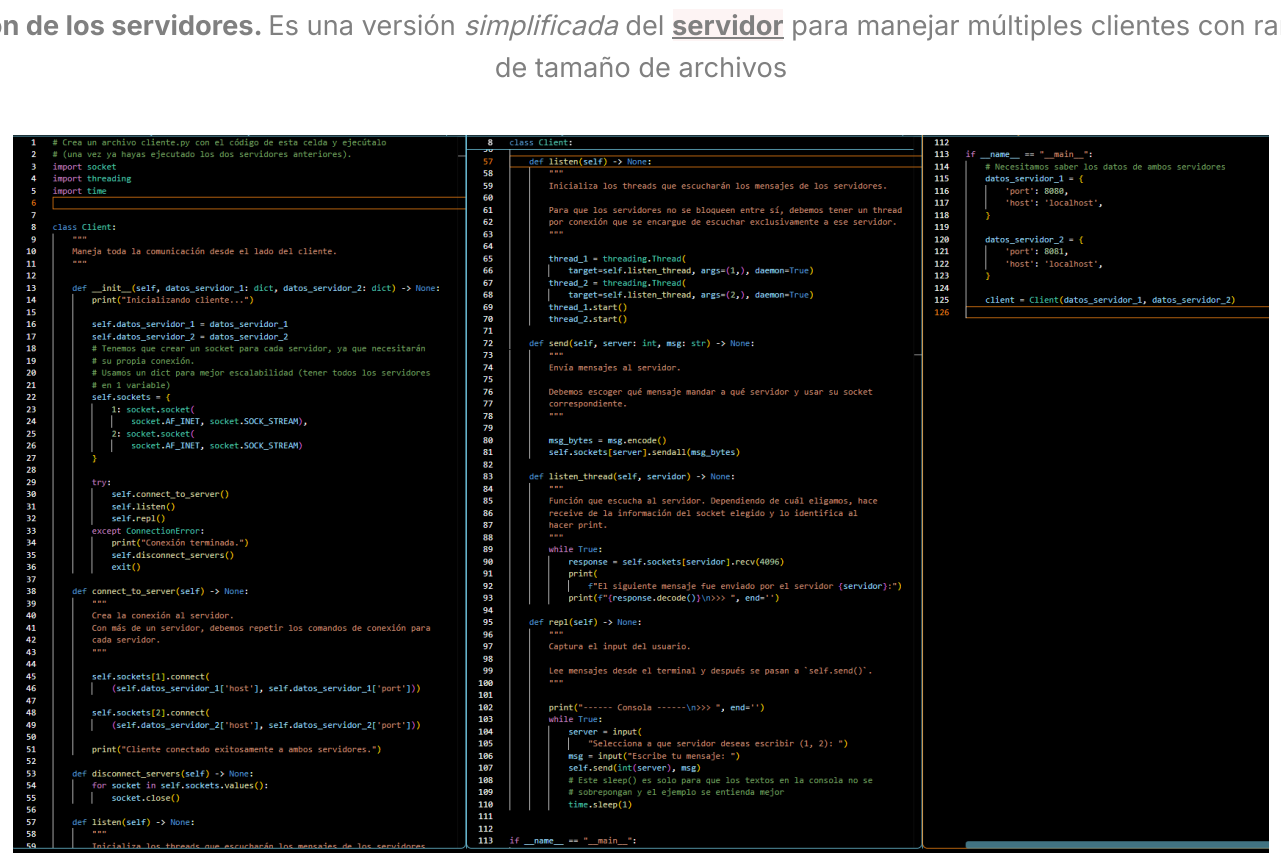
servidor enviando mensajes que pase el usuario y así no detener el flujo principal.

```
# dentro de la clase Cliente
def __init__(...):
    ...
    self.sockets = {} # int: socket
    self.colas = {} # servidor: Queue
def threads_envio(self):
    # ej. un thread de envio por servidor
    for i in self.sockets:
        thread_envio = Thread(target=funcion_envio, args=(i, ), daemon=Tru
e)
        thread_envio.start()
def funcion_envio(self, servidor: int):
        while True:
          item = self.colas[servidor].get()
          self.send(item, servidor)
def send(self, item: any, servidor: int):
    # implementa protocolo de aplicación
    # ej. enviar el tamaño del archivo en los 4 primeros bytes
```

▼ Implementación completa



Configuración de los servidores. Es una versión *simplificada* del **servidor** para manejar múltiples clientes con rango dinámico de tamaño de archivos



Configuración del cliente

▼ Arquitectura P2P (peer-to-peer)

- Puntos clave

- Cada archivo tiene la misma clase Cliente, pero con los datos del otro cliente respectivo para poder conectarse a este.
- En el `__init__` están los datos de la `address` del cliente y de la del otro cliente.

▼ Métodos para hacer de servidor

- `bind_and_listen()`
- `accept_connections()` → crea thread con `target=accept_connections_thread`
- `accept_connections_thread()` → crea un thread para cada cliente con `target=listen_client_thread(client_socket)`
  - En este caso, tiene la particularidad de correr `connect_to_server()` → `listen()` → `repl()` que corresponden a los métodos para hacer de cliente
- `listen_client_thread(client_socket)` → recibe la información del cliente, la decodifica en `received` y se la pasa a `handle_command(received, client_socket)` . Luego, hace `answer_client(response, client_socket)` donde `response` es lo retornado por `handle_command`
- `handle_command(received, client_socket)` se encarga de generar la respuesta en base a lo recibido
- `answer_client(value, sock)` se encarga de enviar el mensaje

▼ Métodos para hacer de cliente

▼ Una vez se pone a escuchar al cliente, significa que este ya abrió sus conexiones y, por ende, se va a tratar de conectar a este.

```
    def accept_connections_thread(self) -> None:
        """
        Levantamos un thread por cliente que se quiere conectar a nosotros.
        """
        print("Cliente P2P (1) aceptando conexiones...")

        while True:
            client_socket, _ = self.sockets['server'].accept()
            listening_client_thread = threading.Thread(
                target=self.listen_client_thread,
                args=(client_socket, ),
                daemon=True)
            listening_client_thread.start()
            # Como ya aceptamos al otro cliente, significa que este ya está
online.
            # Entonces, podemos conectarnos al servidor de este y volvernos
sus clientes
            try:
                self.connect_to_server()
                self.listen()
                self.repl()
            except ConnectionError:
                print("Conexión terminada.")
                self.sockets['client'].close()
                exit()
```

- `connect_to_server()`
- `listen()` → crea thread con `target=listen_thread`
- `listen_thread()` → escucha al otro cliente como si fuera un servidor
- `send(msg)` → envía mensajes al otro cliente como si fuera un servidor
- `repl()` → capta el input del usuario y lo envía al otro cliente vía `send(msg)`

- **Desafío**

- Cliente que hace de servidor para múltiples clientes, y cliente para múltiples servidores → todo en un mismo archivo.

▼ Implementación completa

Esta es la configuración del primer cliente y debe ser ejecutado antes que el segundo cliente. Puesto que, el segundo cliente espera que se abra el primer cliente y el primer cliente espera que se abra el segundo cliente. [La configuración del segundo cliente es igual, sólo se intercambian los puertos]



Sockets UDP

`ejemplo = socket.socket(family, socket.SOCK_DGRAM)`

▼ Implementación cliente y servidor con sockets UDP



Cliente UDP

- No requiere solicitar una conexión previa (no hay un `connect`), pero cada vez que se envía un mensaje se debe especificar la `address` de destino.
- Métodos
  - `ejemplo.` **`sendto`** `(mensaje, address)`
    - Retorna el número de bytes enviados.
  - `data, address = ejemplo.` **`recvfrom`** `(buffer)`
  - `ejemplo.` **`close`** `()`

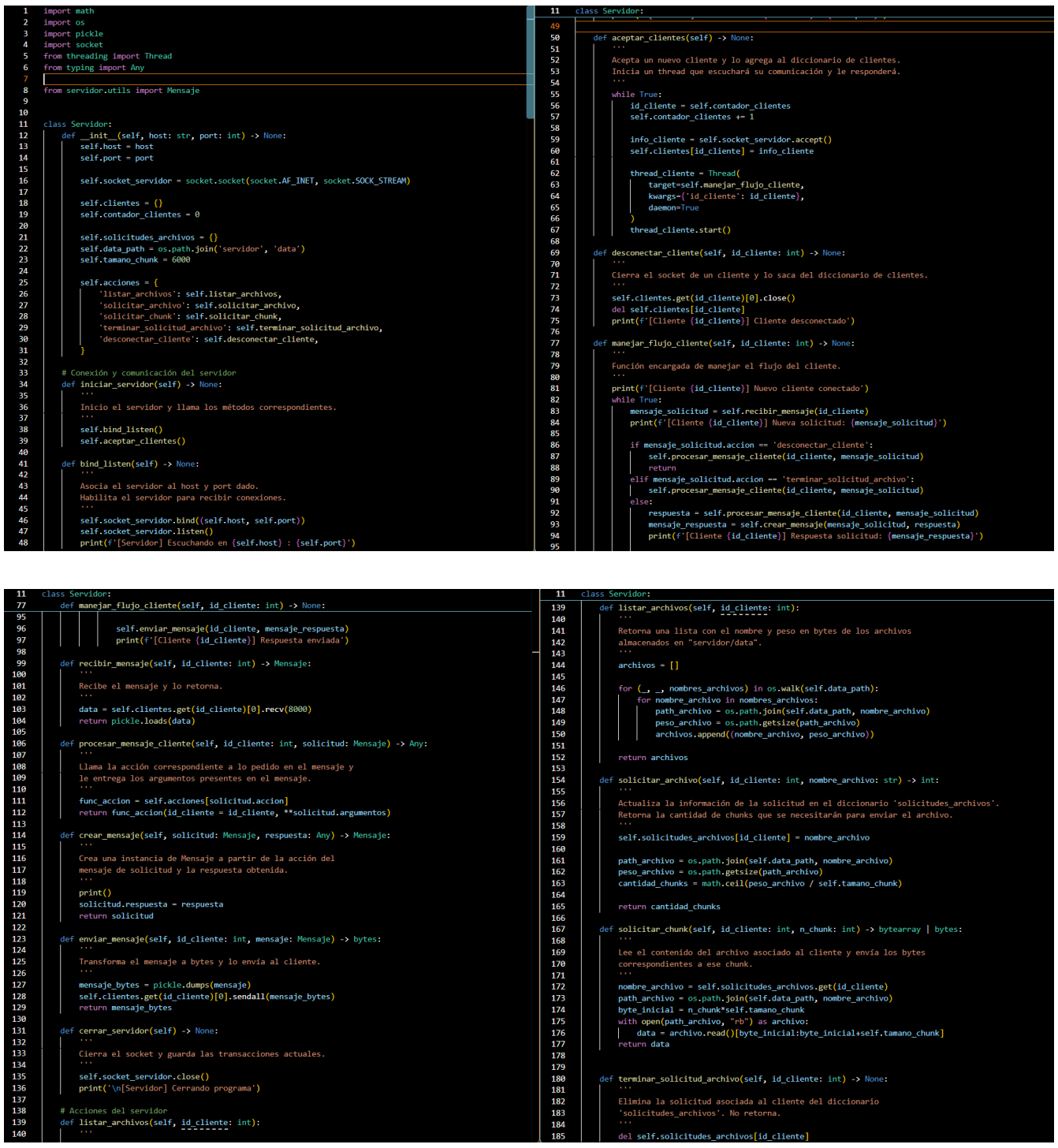
Servidor UDP

- Requiere hacer `ejemplo.` **`bind`** `(address)`

  ▼ En el caso de sockets que usan IPv4, se puede poner `ejemplo.bind(("", port))` donde `""` representa a **INADDR_any** → permite acceder al servidor desde **todas las interfaces de red disponibles en la máquina**
  - ej. IP del Wi-Fi, IP de red local o adaptador interno `127.0.0.1`
  - Por esto en el ejemplo se ocupa `socket.` **`gethostname`** `()` para obtener una interfaz de este tipo.
- No necesita establecer conexión, simplemente se usa `ejemplo.` **`recvfrom`** `(buffer)` para obtener la `data` y la `address` para enviarlo de vuelta.

Ejemplos de actividades pasadas

▼ Servidor con diccionario para gestionar conexiones con clientes



```python
import math
import os
import pickle
import socket
from threading import Thread
from typing import Any

from servidor.utils import Mensaje


class Servidor:
    def __init__(self, host: str, port: int) -> None:
        self.host = host
        self.port = port

        self.socket_servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        self.clientes = {}
        self.contador_clientes = 0

        self.solicitudes_archivos = {}
        self.data_path = os.path.join('servidor', 'data')
        self.tamano_chunk = 6000

        self.acciones = {
            'listar_archivos': self.listar_archivos,
            'solicitar_archivo': self.solicitar_archivo,
            'solicitar_chunk': self.solicitar_chunk,
            'terminar_solicitud_archivo': self.terminar_solicitud_archivo,
            'desconectar_cliente': self.desconectar_cliente,
        }

    # Conexión y comunicación del servidor
    def iniciar_servidor(self) -> None:
        '''
        Inicio el servidor y llama los métodos correspondientes.
        '''
        self.bind_listen()
        self.aceptar_clientes()

    def bind_listen(self) -> None:
        '''
        Asocia el servidor al host y port dado.
        Habilita el servidor para recibir conexiones.
        '''
        self.socket_servidor.bind((self.host, self.port))
        self.socket_servidor.listen()
        print(f'[Servidor] Escuchando en {self.host} : {self.port}')

    def aceptar_clientes(self) -> None:
        '''
        Acepta un nuevo cliente y lo agrega al diccionario de clientes.
        Inicia un thread que escuchará su comunicación y le responderá.
        '''
        while True:
            id_cliente = self.contador_clientes
            self.contador_clientes += 1

            info_cliente = self.socket_servidor.accept()
            self.clientes[id_cliente] = info_cliente

            thread_cliente = Thread(
                target=self.manejar_flujo_cliente,
                kwargs={'id_cliente': id_cliente},
                daemon=True
            )
            thread_cliente.start()

    def desconectar_cliente(self, id_cliente: int) -> None:
        '''
        Cierra el socket de un cliente y lo saca del diccionario de clientes.
        '''
        self.clientes.get(id_cliente)[0].close()
        del self.clientes[id_cliente]
        print(f'[Cliente {id_cliente}] Cliente desconectado')

    def manejar_flujo_cliente(self, id_cliente: int) -> None:
        '''
        Función encargada de manejar el flujo del cliente.
        '''
        print(f'[Cliente {id_cliente}] Nuevo cliente conectado')
        while True:
            mensaje_solicitud = self.recibir_mensaje(id_cliente)
            print(f'[Cliente {id_cliente}] Nueva solicitud: {mensaje_solicitud}')

            if mensaje_solicitud.accion == 'desconectar_cliente':
                self.procesar_mensaje_cliente(id_cliente, mensaje_solicitud)
                return
            elif mensaje_solicitud.accion == 'terminar_solicitud_archivo':
                self.procesar_mensaje_cliente(id_cliente, mensaje_solicitud)
            else:
                respuesta = self.procesar_mensaje_cliente(id_cliente, mensaje_solicitud)
                mensaje_respuesta = self.crear_mensaje(mensaje_solicitud, respuesta)
                print(f'[Cliente {id_cliente}] Respuesta solicitud: {mensaje_respuesta}')
                self.enviar_mensaje(id_cliente, mensaje_respuesta)
                print(f'[Cliente {id_cliente}] Respuesta enviada')

    def recibir_mensaje(self, id_cliente: int) -> Mensaje:
        '''
        Recibe el mensaje y lo retorna.
        '''
        data = self.clientes.get(id_cliente)[0].recv(8000)
        return pickle.loads(data)

    def procesar_mensaje_cliente(self, id_cliente: int, solicitud: Mensaje) -> Any:
        '''
        Llama la acción correspondiente a lo pedido en el mensaje y
        le entrega los argumentos presentes en el mensaje.
        '''
        func_accion = self.acciones[solicitud.accion]
        return func_accion(id_cliente = id_cliente, **solicitud.argumentos)

    def crear_mensaje(self, solicitud: Mensaje, respuesta: Any) -> Mensaje:
        '''
        Crea una instancia de Mensaje a partir de la acción del
        mensaje de solicitud y la respuesta obtenida.
        '''
        print()
        solicitud.respuesta = respuesta
        return solicitud

    def enviar_mensaje(self, id_cliente: int, mensaje: Mensaje) -> bytes:
        '''
        Transforma el mensaje a bytes y lo envía al cliente.
        '''
        mensaje_bytes = pickle.dumps(mensaje)
        self.clientes.get(id_cliente)[0].sendall(mensaje_bytes)
        return mensaje_bytes

    def cerrar_servidor(self) -> None:
        '''
        Cierra el socket y guarda las transacciones actuales.
        '''
        self.socket_servidor.close()
        print('\n[Servidor] Cerrando programa')

    # Acciones del servidor
    def listar_archivos(self, id_cliente: int):
        '''
        Retorna una lista con el nombre y peso en bytes de los archivos
        almacenados en "servidor/data".
        '''
        archivos = []

        for (_, _, nombres_archivos) in os.walk(self.data_path):
            for nombre_archivo in nombres_archivos:
                path_archivo = os.path.join(self.data_path, nombre_archivo)
                peso_archivo = os.path.getsize(path_archivo)
                archivos.append((nombre_archivo, peso_archivo))

        return archivos

    def solicitar_archivo(self, id_cliente: int, nombre_archivo: str) -> int:
        '''
        Actualiza la información de la solicitud en el diccionario 'solicitudes_archivos'.
        Retorna la cantidad de chunks que se necesitarán para enviar el archivo.
        '''
        self.solicitudes_archivos[id_cliente] = nombre_archivo

        path_archivo = os.path.join(self.data_path, nombre_archivo)
        peso_archivo = os.path.getsize(path_archivo)
        cantidad_chunks = math.ceil(peso_archivo / self.tamano_chunk)

        return cantidad_chunks

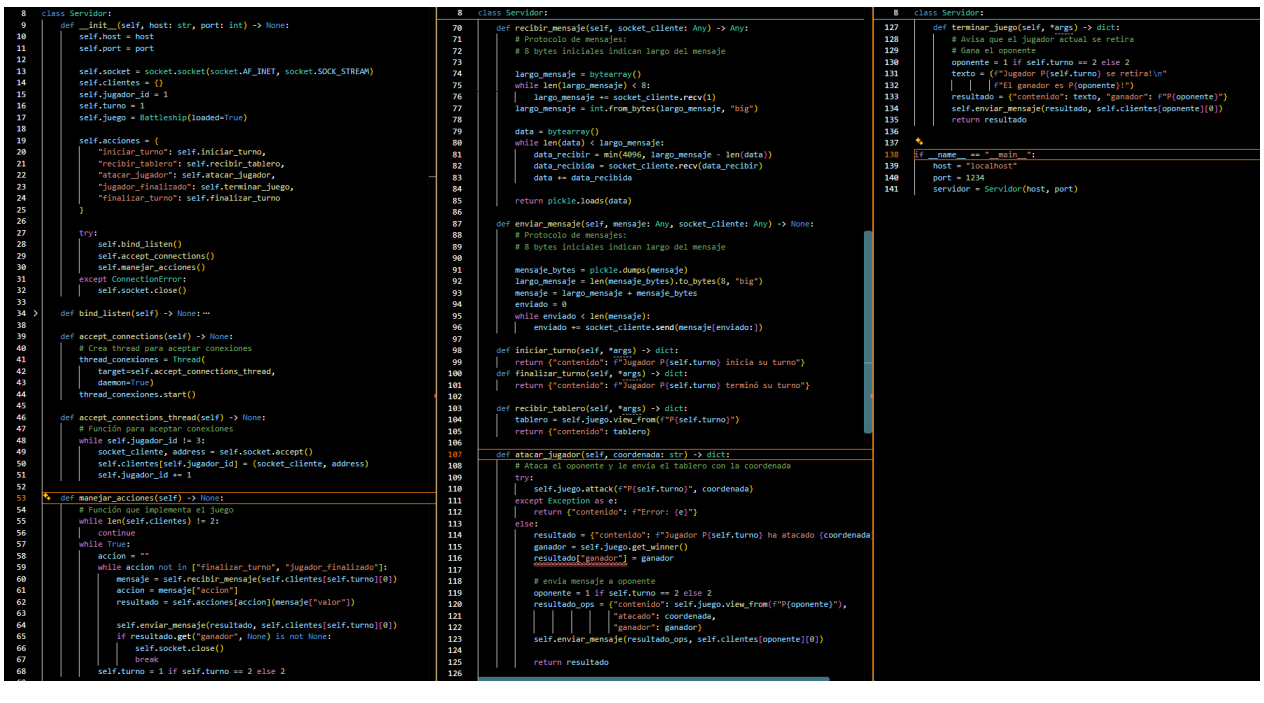
    def solicitar_chunk(self, id_cliente: int, n_chunk: int) -> bytearray | bytes:
        '''
        Lee el contenido del archivo asociado al cliente y envía los bytes
        correspondientes a ese chunk.
        '''
        nombre_archivo = self.solicitudes_archivos.get(id_cliente)
        path_archivo = os.path.join(self.data_path, nombre_archivo)
        byte_inicial = n_chunk*self.tamano_chunk
        with open(path_archivo, "rb") as archivo:
            data = archivo.read()[byte_inicial:byte_inicial+self.tamano_chunk]
        return data


    def terminar_solicitud_archivo(self, id_cliente: int) -> None:
        '''
        Elimina la solicitud asociada al cliente del diccionario
        'solicitudes_archivos'. No retorna.
        '''
        del self.solicitudes_archivos[id_cliente]
```

▼ Battleship

- Puntos clave
  - Manejo de eventos tanto de forma sincrónica como asincrónica → `MainThread` , `Event` , `Lock`
  - Protocolo de mensajes
  - Servidor que maneja múltiples clientes
  - Cliente con threads individuales para recibir/mandar mensajes (modelo Productor-Consumidor).
  - Cierre de servidor y fin del programa

▼ v1.0



Configuración servidor



Configuración cliente

▼ Observaciones

▼ Solamente el MainThread es capaz de finalizar el programa

- En este caso, `Client.py` puede finalizar tanto de manera sincrónica (ejecutando la lógica del juego) como asincrónica (por salida del otro cliente).
- Bajo la lógica de v1.0, el `input` bloquea el código y el jugador debe apretar `Enter` para que en la siguiente iteración termine el juego.
- Para evitar que se ejecute código después que llega el aviso del otro cliente, MainThread debe estar centrado en esperar dicho mensaje y finalizar el programa. Por lo tanto, toda la funcionalidad debería estar organizada por otro “thread principal”.

▼ `server.py` no permite manejar múltiples clientes a la vez

- Establece un bloqueo estricto entre clientes, lo cuál no permite comunicar mensajes asincrónicos (ej. un cliente se desconectó).
- Idealmente debería haber un flujo independiente para cada cliente (`Thread`), pero distinguiendo entre eventos que sólo pueden ocurrir durante el turno de un cliente, de aquellos eventos que pueden ocurrir en cualquier momento → `Lock`
- Al implementar la lógica por cliente, todas las funciones del juego deberían funcionar con respecto al `jugador_id` en lugar de `self.turno`
  - En el caso de `client.py` , se debería implementar lógica para pedirle el turno al servidor y recién ahí comenzar las acciones.

▼ La condición de inicio de juego consume mucha CPU

- Para comenzar el juego se deben conectar 2 clientes y esto hace que se rompa el loop `while len(self.clientes) != 2` . Sin embargo, esto revisa constantemente si se cumple dicha condición.
- Alternativa → `event.wait()`

▼ `client.py` no permite realizar acciones asincrónicas

- Debería poder recibir un mensaje del servidor y realizar algo, mientras no sea el turno del jugador asociado a ese cliente.
- Propuesta → Agregar bloques `if` en `listen_thread()`

▼ v1.1

▼ Servidor

```python
import socket
from threading import Thread, Lock, Event
import sys
from battleship import Battleship
from typing import Any
import pickle

class Servidor:
    def __init__(self, host: str, port: int) -> None:
        self.host = host
        self.port = port

        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.clientes = {}
        self.jugador_id = 1
        self.turno = 1
        self.lock = Lock()
        self.juego = Battleship(loaded=True)
        self.empezo_juego = Event()
        self.termino_juego = Event()

        try:
            self.bind_listen()
            self.accept_connections()
            self.termino_juego.wait()
        except ConnectionError:
            self.socket.close()

    def bind_listen(self) -> None:
        # Enlaza servidor a una address y lo abre
        self.socket.bind((self.host, self.port))
        self.socket.listen()

    def accept_connections(self) -> None:
        # Crea thread para aceptar conexiones
        thread_conexiones = Thread(
            target=self.accept_connections_thread,
            daemon=True)
        thread_conexiones.start()

    def accept_connections_thread(self) -> None:
        # Función para aceptar conexiones
        # crea un thread para cada cliente
        while self.jugador_id != 3:
            socket_cliente, address = self.socket.accept()
            self.clientes[self.jugador_id] = (socket_cliente, address)

            thread_cliente = Thread(target=self.manejar_acciones,
                                    args=(self.jugador_id, ), daemon=True)
            thread_cliente.start()

            self.jugador_id += 1
        self.empezar_juego()

    def manejar_acciones(self, jugador_id: int) -> None:
        # Función que implementa el juego para cada jugador
        self.empezo_juego.wait()
        while True:
            try:
                mensaje = self.recibir_mensaje(self.clientes[jugador_id][0])
            except ConnectionError:
                self.terminar_juego(jugador_id)
                break
            except (OSError, EOFError):
                break
            else:
                accion = mensaje["accion"]
                # Acción asincrónica:
                if accion == "jugador_finalizado":
                    self.terminar_juego(jugador_id)
                    break
                elif accion == "fin":
                    self.socket.close()
                    break
                # Acciones sincrónicas:
                else:
                    with self.lock:
                        if self.turno != jugador_id:
                            continue
                        if accion == "recibir_tablero":
                            resultado = self.recibir_tablero(jugador_id)
                        elif accion == "finalizar_turno":
                            resultado = self.finalizar_turno(jugador_id)
                        else:
                            resultado = self.atacar_jugador(jugador_id, mensaje["valor"])
                        self.enviar_mensaje(resultado, self.clientes[jugador_id][0])


    def recibir_mensaje(self, socket_cliente: Any) -> Any:
        # Protocolo de mensajes:
        # 8 bytes iniciales indican largo del mensaje

        largo_mensaje = bytearray()
        while len(largo_mensaje) < 8:
            byte = socket_cliente.recv(1)
            if not byte:
                raise ConnectionError()
            largo_mensaje += byte
        largo_mensaje = int.from_bytes(largo_mensaje, "big")

        data = bytearray()
        while len(data) < largo_mensaje:
            data_recibir = min(4096, largo_mensaje - len(data))
            data_recibida = socket_cliente.recv(data_recibir)
            data += data_recibida

        return pickle.loads(data)

    def enviar_mensaje(self, mensaje: Any, socket_cliente: Any) -> None:
        # Protocolo de mensajes:
        # 8 bytes iniciales indican largo del mensaje

        mensaje_bytes = pickle.dumps(mensaje)
        largo_mensaje = len(mensaje_bytes).to_bytes(8, "big")
        mensaje = largo_mensaje + mensaje_bytes
        enviado = 0
        try:
            while enviado < len(mensaje):
                enviado += socket_cliente.send(mensaje[enviado:])
        except (ConnectionError, OSError, EOFError):
            pass

    def finalizar_turno(self, jugador_id: int) -> dict:
        self.turno = 1 if jugador_id == 2 else 2
        self.enviar_mensaje({"turno": "Es tu turno"}, self.clientes[self.turno][0])
        return {"contenido": f"Jugador P{jugador_id}, terminó tu turno"}

    def recibir_tablero(self, jugador_id: int) -> dict:
        tablero = self.juego.view_from(f"P{jugador_id}")
        return {"contenido": tablero}

    def atacar_jugador(self, jugador_id: int, coordenada: str) -> dict:
        # Ataca el oponente y le envía el tablero con la coordenada
        try:
            self.juego.attack(f"P{jugador_id}", coordenada)
        except Exception as e:
            return {"contenido": f"Error: {e}"}
        else:
            resultado = {"contenido": f"Jugador P{jugador_id} ha atacado {coordenada}"}
            ganador = self.juego.get_winner()
            resultado["ganador"] = ganador
            if ganador is not None:
                resultado["contenido"] = (resultado["contenido"]+
                                          f"\nHa ganado {ganador}")

            # envia mensaje a oponente
            oponente = 1 if jugador_id == 2 else 2
            resultado_ops = {"contenido": self.juego.view_from(f"P{oponente}"),
                             "atacado": f"Has sido atacado en {coordenada}!\n",
                             "ganador": ganador}
            self.enviar_mensaje(resultado_ops, self.clientes[oponente][0])

            return resultado

    def empezar_juego(self) -> None:
        self.enviar_mensaje({"turno": "Es tu turno"}, self.clientes[1][0])
        self.empezo_juego.set()

    def terminar_juego(self, jugador_id: int) -> None:
        # Avisa que el jugador actual se retira
        # Gana el oponente
        oponente = 1 if jugador_id == 2 else 2
        texto = (f"Jugador P{jugador_id} se retira!\n"
                 f"El ganador es P{oponente}!")
        resultado = {"contenido": texto, "ganador": f"P{oponente}"}

        for id in [oponente, jugador_id]:
            self.enviar_mensaje(resultado, self.clientes[id][0])
        self.termino_juego.set()


if __name__ == "__main__":
    host = "localhost"
    port = 1234
    servidor = Servidor(host, port)
```

▼ Cliente

```python
from threading import Thread, Event
import socket
import sys
from typing import Any
import pickle
from queue import Queue

class Cliente:
    def __init__(self, host: str, port: int) -> None:
        self.host = host
        self.port = port

        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.salidas = Queue()
        self.entradas = Queue()
        self.juego_terminado = Event()
        self.mi_turno = Event()

        try:
            self.socket.connect((self.host, self.port))
            self.listen()
            self.send()
        except ConnectionError:
            self.socket.close()

    def listen(self) -> None:
        # Crea un thread para el servidor
        thread_servidor = Thread(target=self.listen_thread, daemon=True)
        thread_servidor.start()

    def listen_thread(self) -> None:
        # Recibe mensajes desde el servidor
        while True:
            try:
                mensaje = self.recibir_mensaje()
            except (ConnectionError, EOFError, OSError):
                self.socket.close()
                self.juego_terminado.set()
                break

            # Se encarga de comenzar el turno
            if mensaje.get("turno", None) is not None:
                self.mi_turno.set()
                print(mensaje["turno"])

            # Se encarga de mostrar el tablero cuando es atacado
            elif mensaje.get("atacado", None) is not None:
                print(f"{mensaje["atacado"]}")
                print(mensaje["contenido"])
                if mensaje.get("ganador", None) is not None:
                    print("Se acabó el juego\n"
                          f"El ganador es {mensaje["ganador"]}")
                    self.juego_terminado.set()

            # Se encarga de terminar el juego de forma asincrónica
            elif mensaje.get("ganador", None) is not None:
                print(mensaje["contenido"])
                self.juego_terminado.set()

            # Se encarga de manejar el turno actual
            else:
                self.entradas.put(mensaje)

    def recibir_mensaje(self) -> Any:
        # Protocolo de mensajes:
        # 8 bytes iniciales indican largo del mensaje

        largo_mensaje = bytearray()
        while len(largo_mensaje) < 8:
            byte = self.socket.recv(1)
            if not byte:
                raise ConnectionError()
            largo_mensaje += byte
        largo_mensaje = int.from_bytes(largo_mensaje, "big")

        data = bytearray()
        while len(data) < largo_mensaje:
            data_recibir = min(4096, largo_mensaje - len(data))
            data_recibida = self.socket.recv(data_recibir)
            data += data_recibida

        return pickle.loads(data)

    def send(self) -> None:
        # Crea thread para enviarle mensajes al servidor
        thread_mensajes = Thread(target=self.send_thread, daemon=True)
        thread_mensajes.start()

    def send_thread(self) -> None:
        # Envia mensajes al servidor
        while True:
            mensaje = self.salidas.get()
            self.enviar_mensaje(mensaje)
            self.salidas.task_done()

    def enviar_mensaje(self, mensaje: Any) -> None:
        # Protocolo de mensajes:
        # 8 bytes iniciales indican largo del mensaje

        mensaje_bytes = pickle.dumps(mensaje)
        largo_mensaje = len(mensaje_bytes).to_bytes(8, "big")
        mensaje = largo_mensaje + mensaje_bytes
        enviado = 0
        while enviado < len(mensaje):
            enviado += self.socket.send(mensaje[enviado:])

    def accion(self, accion: str, valor: Any = None) -> bool:
        self.salidas.put({"accion": accion, "valor": valor})
        return self.obtener_respuesta()

    def obtener_respuesta(self) -> bool:
        # Imprime la respuesta del servidor
        # e indica si ya terminó el juego
        datos = self.entradas.get()
        print(datos["contenido"])
        ganador = datos.get("ganador", None)
        self.entradas.task_done()

        if ganador is not None:
            print(f"Se acabó el juego.\n"
                  f"Ganó {ganador}\n")
            self.enviar_mensaje({"accion": "fin"})
            return True
        return False

def jugar(cliente: Cliente) -> None:
    opciones = ("Seleccione una opcion:\n"
        "[1] Atacar jugador\n"
        "[2] Salir\n")
    while True:
        print("\n\n")
        cliente.mi_turno.wait()
        cliente.accion("recibir_tablero")
        opcion = input(opciones)
        while opcion not in ["1", "2"]:
            print("Opcion invalida")
            opcion = input(opciones)
        if opcion == "1":
            coordenada = input("Ingrese coordenada a atacar: ")
            if cliente.accion("atacar_jugador", coordenada):
                cliente.juego_terminado.set()
            cliente.accion("recibir_tablero")
            cliente.accion("finalizar_turno")
            cliente.mi_turno.clear()
        elif opcion == "2":
            cliente.accion("jugador_finalizado")
            break

if __name__ == "__main__":
    host = "localhost"
    port = 1234
    cliente = Cliente(host, port)
    thread_juego = Thread(target=jugar, args=(cliente, ), daemon=True)
    thread_juego.start()
    cliente.juego_terminado.wait()
```